Operating System Principles

Module – 3

Process Scheduling

- The objective of multiprogramming is to have some process running at all times so as to maximize CPU utilization.
- The objective of time sharing is to switch a CPU core among processes so frequently that users can interact with each program while it is running.
- To meet these objectives, the **process scheduler** was used to select an available process for program execution.

Process Scheduling

- The number of processes currently in memory is known as the **degree of multiprogramming**.
- An **I/O-bound process** is one that spends more of its time doing I/O than it spends doing computations.
- A **CPU-bound process**, in contrast, generates I/O requests infrequently, using more of its time doing computations.

Scheduling Queues

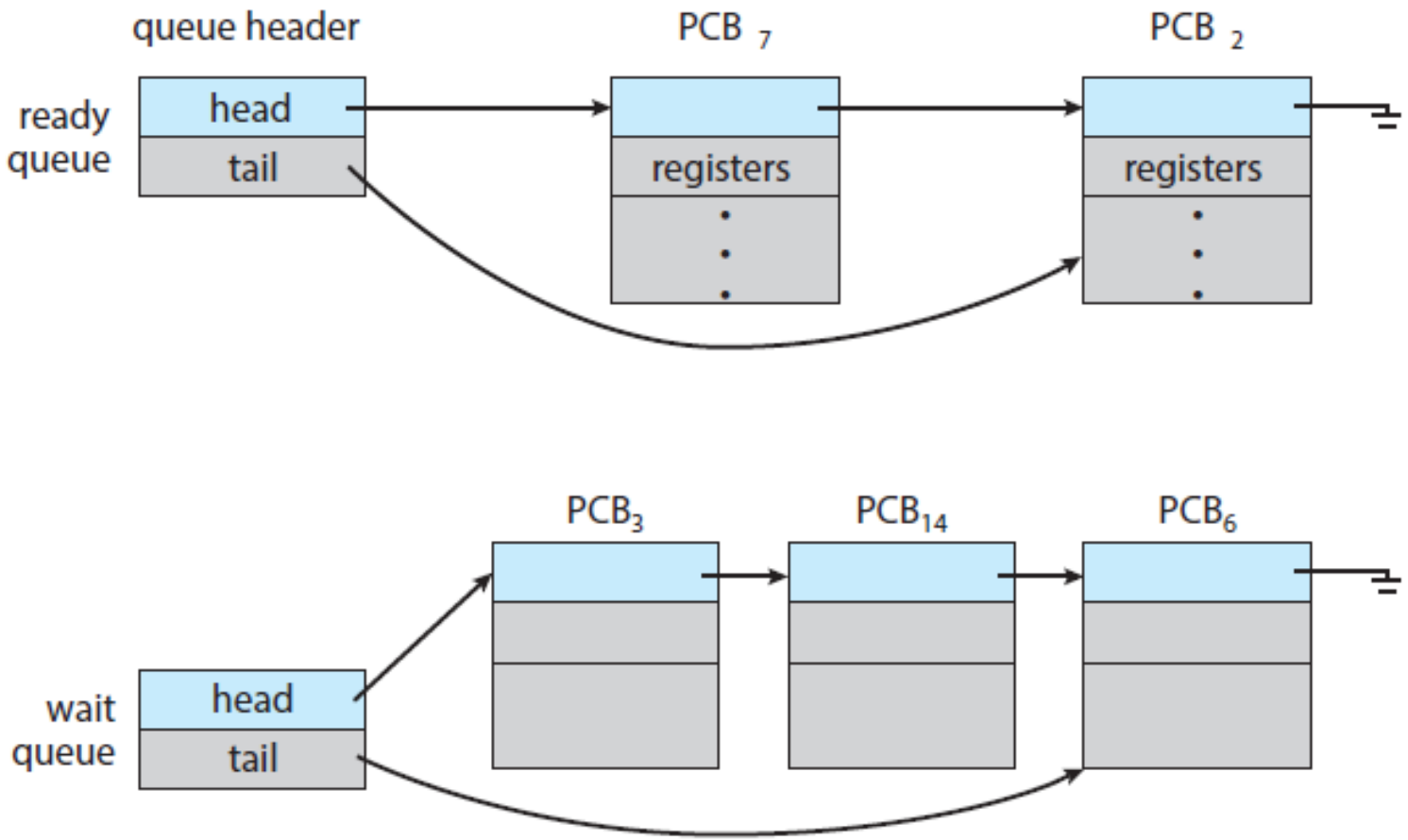


Figure: The ready queue and wait queues.

Scheduling Queues

- As processes enter the system, they are put into a **ready queue**, where they are ready and waiting to execute on a CPU's core.
- This queue is generally stored as a linked list; a ready-queue header contains pointers to the first PCB in the list, and each PCB includes a pointer field that points to the next PCB in the ready queue.

Scheduling Queues

- When a process is allocated a CPU core, it executes for a while and eventually terminates, is interrupted, or waits for the occurrence of a particular event, such as the completion of an I/O request.
- Processes that are waiting for a certain event to occur — such as completion of I/O — are placed in a **wait queue**

Scheduling Queues

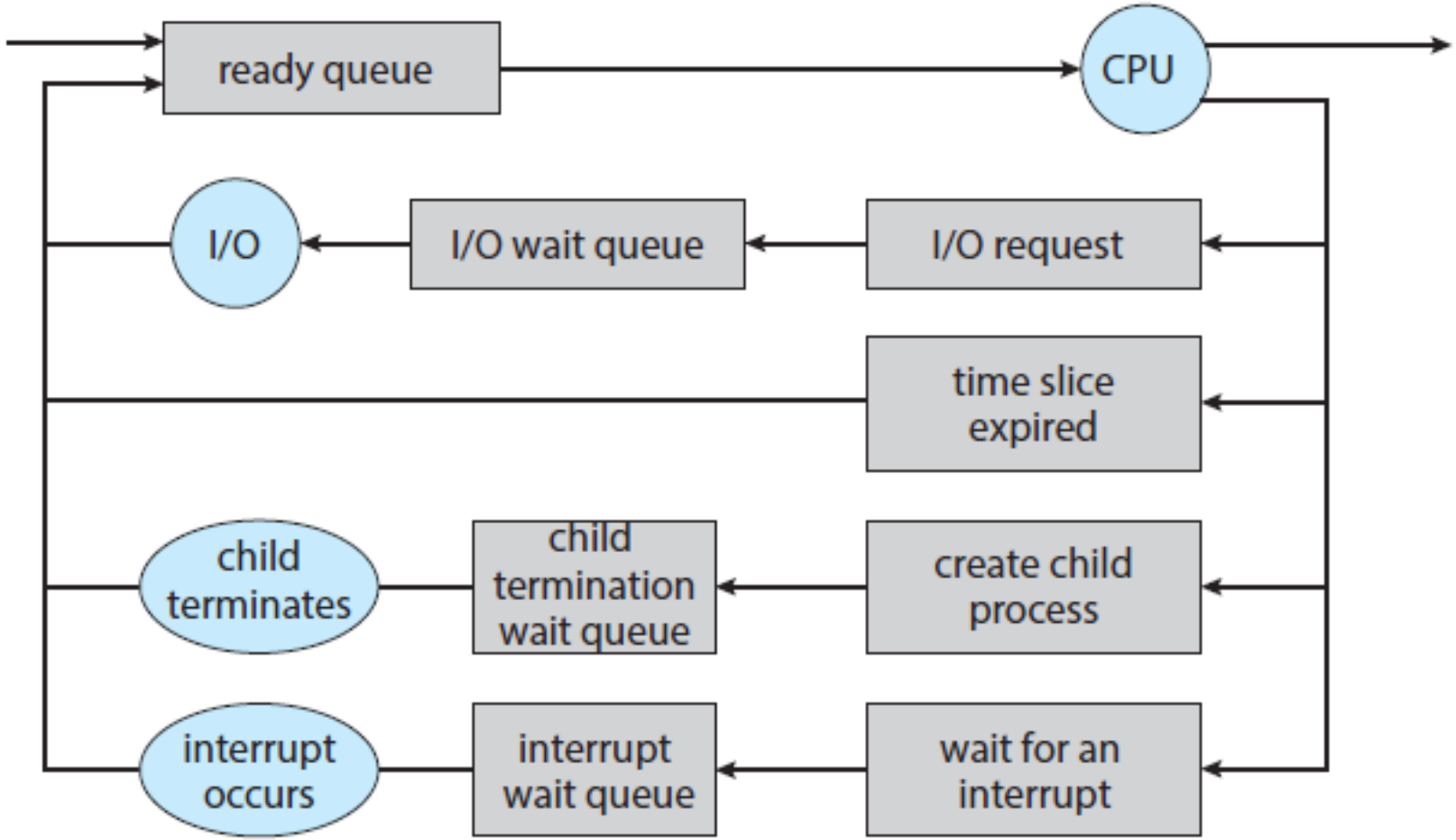


Figure: Queuing-diagram representation of process scheduling.

CPU Scheduling

- A process migrates among the ready queue and various wait queues throughout its lifetime.
- The role of the **CPU scheduler** is to select from among the processes that are in the ready queue and allocate a CPU core to one of them

CPU Scheduling

Swapping **-** a process can be "swapped out" from memory to disk, where its current status is saved, and later "swapped in" from disk back to memory, where its status is restored.

- Swapping is typically only necessary when memory has been overcommitted and must be freed up

Context Switch

- Interrupts cause the operating system to change a CPU core from its current task and to run a kernel routine.
- When an interrupt occurs, the system needs to save the current **context** of the process running on the CPU core so that it can restore that context when its processing is done.
- **context switch: state save to state restore**

Context Switch

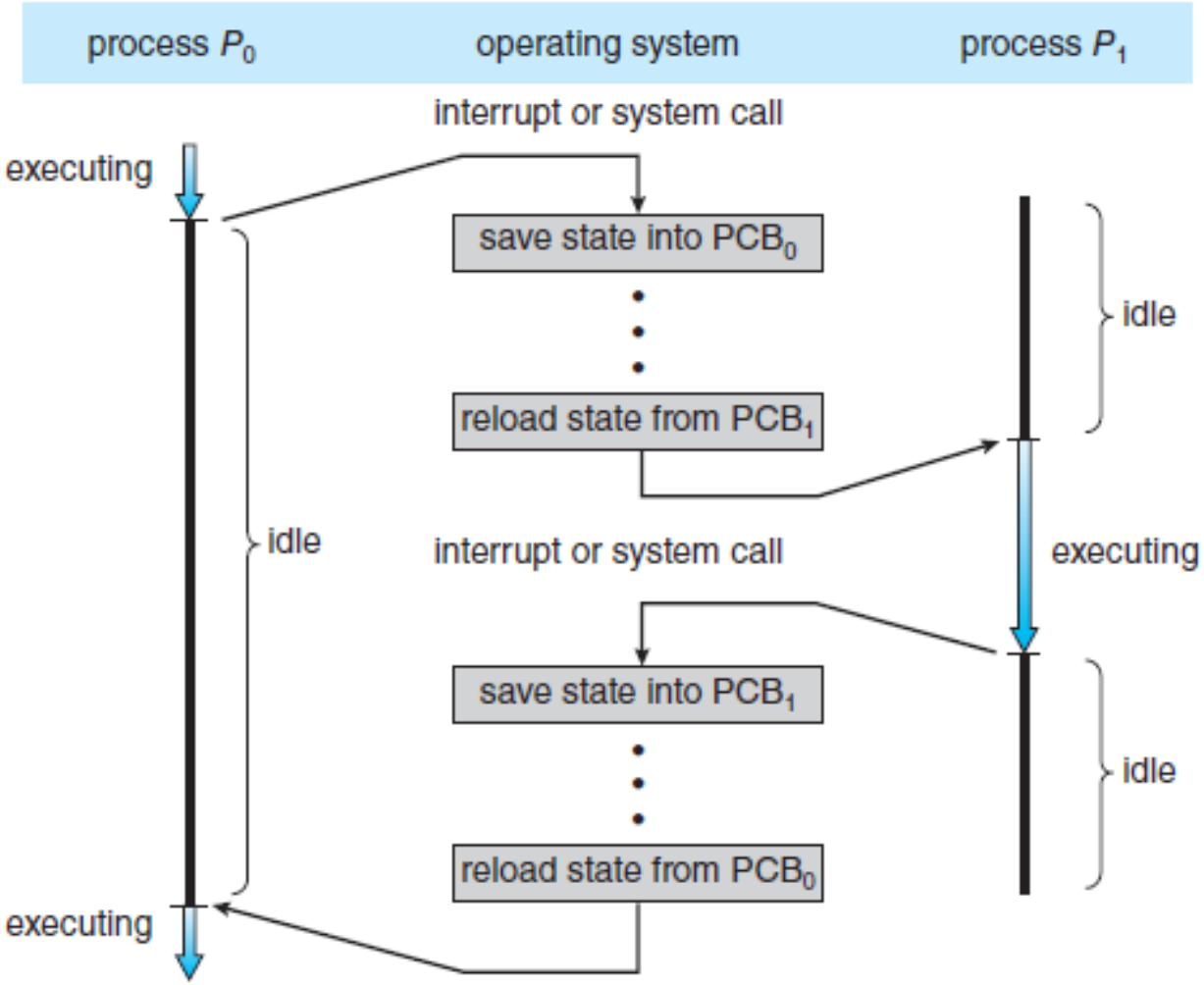


Figure: Diagram showing context switch from process to process

CPU Scheduling

- The operating system schedules the processes in such a way that the CPU doesn't sit idle and keeps processing some or the other process.

- Process scheduling is a technique used by operating systems to manage the execution of multiple processes that are competing for the same resources, such as the CPU (Central Processing Unit), memory, and I/O devices.

- It determines which process should be executed next, based on various criteria such as priority, resource requirements, time limits, and waiting time.

CPU Scheduling

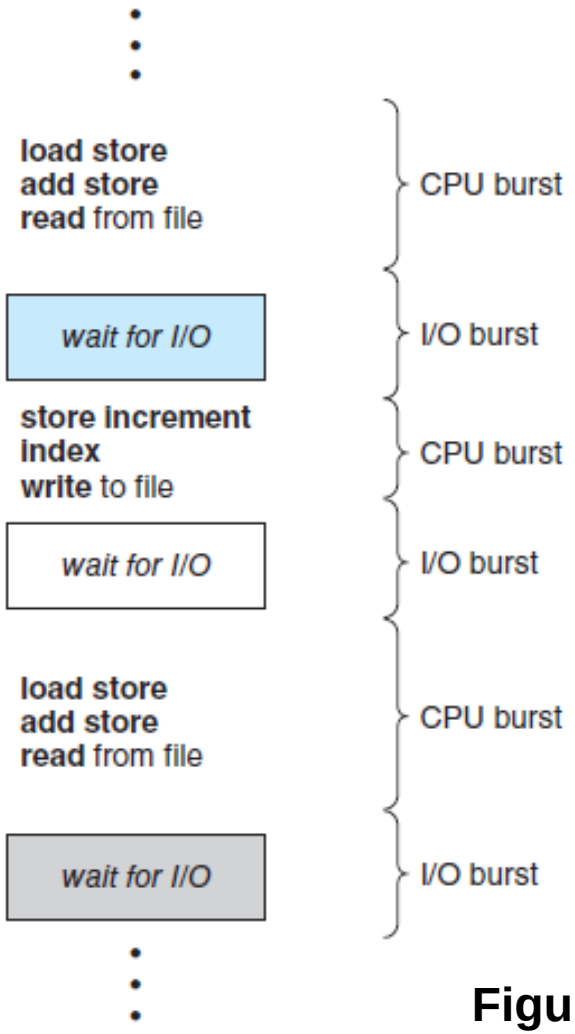


Figure: Alternating sequence of CPU and I/O bursts.

Process Scheduling – Measures & Metrics

1. CPU Burst Time:

- This is the *amount of time a process spends executing on the CPU* without being interrupted. During a CPU burst, the process performs computations, calculations, or any tasks that require the processor's attention.
- CPU bursts can *vary in length depending on the nature of the process*. For example, a computationally intensive task might have long CPU bursts, while a process that frequently requires I/O operations might have shorter CPU bursts.
- CPU burst time is critical for scheduling algorithms to determine when to switch processes to optimize CPU utilization and responsiveness.

Process Scheduling – Measures & Metrics

2. I/O Burst Time

- This refers to *the time a process spends waiting for or performing input/output operations*, such as reading from or writing to a disk, accessing network resources, or interacting with peripheral devices.
- *During an I/O burst*, the *process does not use the CPU*, allowing the CPU to execute other processes. This makes I/O burst time an essential factor in the efficient utilization of system resources.
- *I/O bursts are typically followed by CPU bursts* as processes alternate between computation and I/O activities

Process Scheduling – Measures & Metrics

3. Turnaround Time (TAT):

- The *total time taken for a process to complete from the time it is submitted until it finishes*. It includes waiting time, execution time, and any time spent in the ready queue or waiting for I/O.

Turnaround Time = Completion Time - Arrival Time

Process Scheduling – Measures & Metrics

4. Waiting Time (WT):

- The *total time a process spends waiting* in the *ready queue before* it gets *CPU time*. It does not include the time spent on executing the process or performing I/O.

Waiting Time = Turnaround Time - Burst Time

Process Scheduling – Measures & Metrics

5. Response Time (RT):

- The *time from when a process is submitted until the first response is produced*. For interactive processes, this is an important measure of responsiveness.

Response Time = First Response Time - Arrival Time

- For some algorithms, the response time may be the same as the waiting time, especially in non-preemptive scheduling.

Process Scheduling – Measures & Metrics

6. Completion Time:

- The completion time of a process is the *exact time at which a process finishes its execution and exits* the system.
- It is used to calculate other important metrics such as Turnaround Time and Waiting Time.
- If a process starts executing at time 0 and finishes at time 5, its completion time is 5.

Process Scheduling – Measures & Metrics

7. Arrival Time:

- The arrival time of a process is the *exact time at which a process enters the ready queue and becomes available for execution*.
- This is the time the process starts waiting for the CPU to be allocated to it. It's also used in calculating Turnaround Time, Waiting Time, and Response Time.
- If a process arrives at time 2, then its arrival time is 2.

Preemptive and Non-preemptive Scheduling

CPU-scheduling decisions may take place under the following four circumstances:

1. When a process switches from the running state to the waiting state (for example, as the result of an I/O request or an invocation of wait() for the termination of a child process)

2. When a process switches from the running state to the ready state (for example, when an interrupt occurs)

3. When a process switches from the waiting state to the ready state (for example, at completion of I/O)

4. When a process terminates

Preemptive and Non-preemptive Scheduling

- When scheduling takes place only under circumstances 1 and 4,we say that the scheduling scheme is **non-preemptive** or **cooperative**. Otherwise, it is **preemptive**.
- Under non-preemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state.

Preemptive and Non-preemptive Scheduling

- Virtually all modern operating systems including Windows, macOS, Linux, and UNIX use preemptive scheduling algorithms.
- Preemptive scheduling can result in race conditions when data are shared among several processes

Process Scheduling – First Come First Serve

- It *schedules processes based on the order in which they arrive in the ready queue*.
- The process that arrives first gets executed first, and subsequent processes wait until the CPU becomes available.
- It is **Non-Preemptive.** Once a process starts executing, it runs to completion without being interrupted. Other processes must wait until the current process finishes.

First Come First Serve – Problem 1

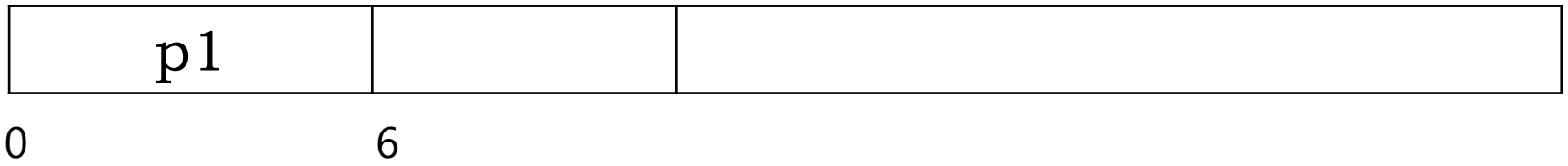
Consider Three Process **p1, p2 and p3** which arrives in the system at time 0. Following are their respective burst time – **6, 3, and 8.**

Process Scheduling – First Come First Serve

- Given:

Process	Arrival Time	Burst Time
p1	0	6
p2	0	3
p3	0	8

- Construct the Gantt Chart

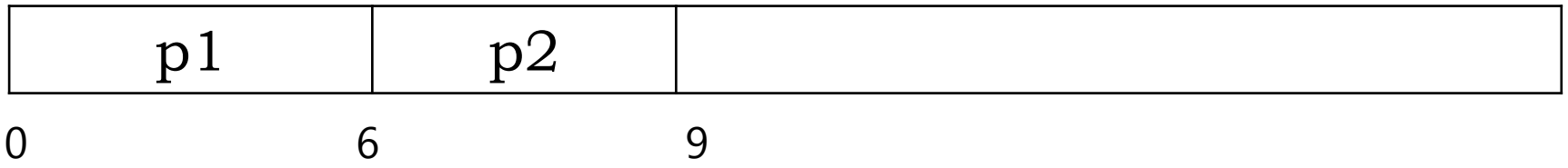


Process Scheduling – First Come First Serve

- Given:

Process	Arrival Time	Burst Time
p1	0	6
p2	0	3
p3	0	8

- Construct the Gantt Chart

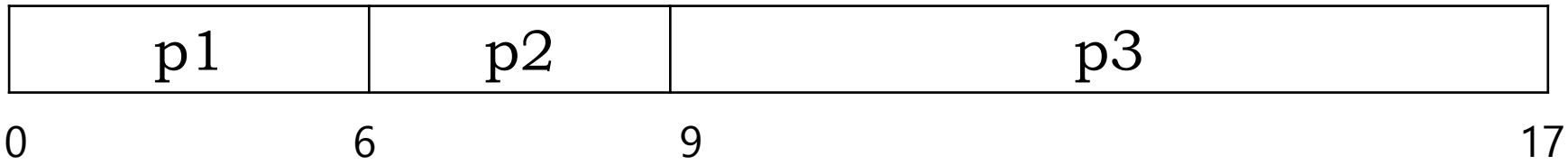


Process Scheduling – First Come First Serve

Given:

Process	Arrival Time	Burst Time
p1	0	6
p2	0	3
p3	0	8

- Construct the Gantt Chart



Process Scheduling – First Come First Serve

- Calculations:

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
p1	0	6	6	6 – 0 = 6	6 – 6 = 0
p2	0	3	9	9 – 0 = 9	9 – 3 = 6
p3	0	8	17	17 – 0 = 17	17 – 8 = 9
Average Turnaround Time				(6 + 9 + 17)/3 = **10.67**	
Average Waiting Time					(0 + 6 + 9)/3 = **5**

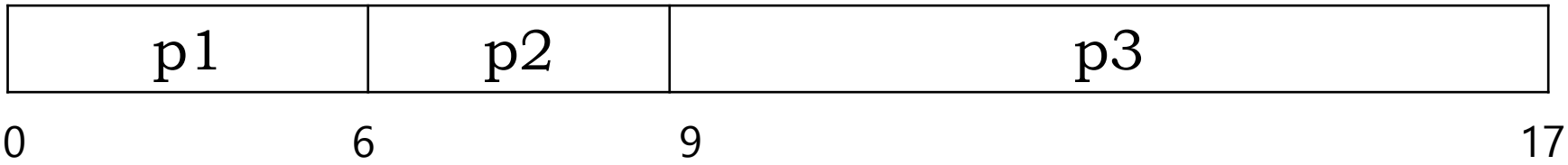
First Come First Serve - Problem 2

Given the processes, find the completion time, turnaround time and waiting time

Process	Arrival Time	Burst Time
p1	0	6
p2	4	3
p3	6	8

Process Scheduling – First Come First Serve

- Construct the Gantt Chart



Process Scheduling – First Come First Serve

- Calculations:

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
p1	0	6	6	6 – 0 = 6	6 – 6 = 0
p2	4	3	9	9 – 4 = 5	5 – 3 = 2
p3	6	8	17	17 – 6 = 11	11 – 8 = 3
Average Turnaround Time				(6 + 5 + 11)/3 = 7.33	
Average Waiting Time					(0 + 2 + 3)/3 = 1.67

First Come First Serve - Problem 3

Given the processes, find the completion time, average turnaround time and average waiting time.

Process	Arrival Time	Burst Time 1	I/O	Burst Time 2
p1	0	4	5	3
p2	1	3	4	2
p3	2	2	3	4

First Come First Serve – Problem for Practice.

1. Consider Three Process **p1, p2 and p3** which arrives in the system at time 0. Following are their respective burst time **24, 3, and 3.**
2. Consider Three Process **p1, p2 and p3** which arrives in the system at time 0. Following are their respective burst time **3, 3, and 24.**

First Come First Serve - Features

- **Non-Preemptive** Once a process starts executing, it runs to completion without being interrupted. Other processes must wait until the current process finishes.
- **Queue-Based**: Processes are managed in a FIFO (First-In, First-Out) queue. The first process to enter the queue is the first to be executed.
- **Simple Implementation**: FCFS is easy to implement because it doesn't require complex decision-making. The next process to execute is always the one at the front of the queue.
- **Fairness:** Each process is treated equally, as they are scheduled in the order they arrive.

First Come First Serve - Drawbacks

- **Convoy Effect**: If a long process is at the front of the queue, it can cause all other processes to wait, leading to inefficiencies. This is known as the "convoy effect."
- **High Average Waiting Time**: Since processes are not scheduled based on their length or urgency, shorter processes may have to wait behind longer ones, resulting in increased average waiting times.
- **Non-Preemptive Nature**: It is *not suitable for systems requiring quick responsiveness*, as once a process starts executing, it cannot be interrupted.

Process Scheduling – Shortest Job First

- A process scheduling algorithm that *selects the process with the* ***shortest burst time*** (the next shortest amount of CPU time needed) for execution next.
- It is also known as Shortest Job Next (SJN) or Shortest Process Next (SPN).
- SJF can be either *preemptive or non-preemptive*.

Process Scheduling – Shortest Job First

- **Non-Preemptive SJF:**
  - In this variant, once a process starts executing, it cannot be stopped until it finishes.
  - The scheduler *selects the process with the shortest burst time among the processes* that have *arrived and are ready to execute*.

Process Scheduling – Shortest Job First

- **Preemptive SJF:**
  - The *scheduler may preempt the currently running process if a new process arrives with a shorter burst time than the remaining time of the current process*.
  - This ensures that the process with the *shortest remaining burst time is always executed*

Process Scheduling – Shortest Job First

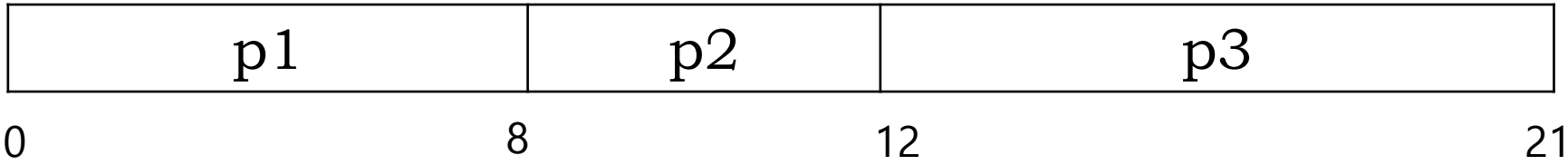
- ***Optimal for Minimizing Average Waiting Time***: SJF is optimal in terms of minimizing the average waiting time for a set of processes, as it schedules shorter jobs first, reducing the waiting time for subsequent processes.
- ***Possibility of Starvation***: Longer processes may experience indefinite postponement if there is a continuous stream of shorter processes arriving. This situation is known as starvation.
- ***Requires Knowledge of Burst Time***: To implement SJF effectively, the exact or estimated burst time of each process must be known in advance, which is not always feasible in real-world scenarios

Process Scheduling – Shortest Job First

- Given

Process	Arrival Time	Burst Time
p1	0	8
p2	1	4
p3	2	9

- Constructing Gantt



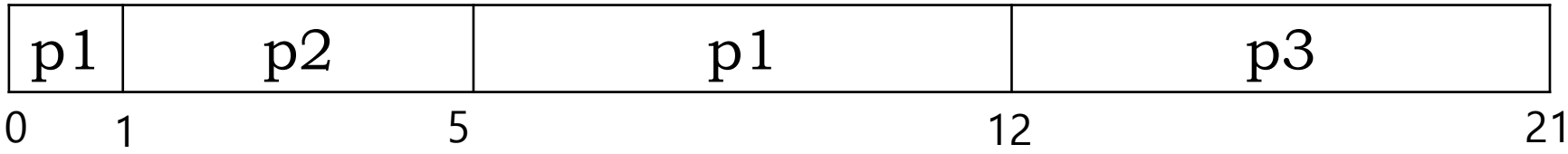
Process Scheduling – Shortest Job First

- Calculations: Non-Preemptive

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
p1	0	8	8	8 – 0 = 8	8 – 8 = 0
p2	1	4	12	12 – 1 = 11	11 – 4 = 7
p3	2	9	21	21 – 2 = 19	19 – 9 = 10
Average Turnaround Time				(8 + 11 + 19)/3 = 12.67	
Average Waiting Time					(0 + 7 + 10)/3 = 5.67

Process Scheduling – Shortest Job First

- Calculations: Preemptive



Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
p1	0	8	12	12 – 0 = 12	12 – 8 = 4
p2	1	4	5	5 – 1 = 4	4 – 4 = 0
p3	2	9	21	21 – 2 = 19	19 – 9 = 10
Average Turnaround Time				(8 + 11 + 19)/3 = 11.67	
Average Waiting Time					(4 + 0 + 10)/3 = 4.67

Process Scheduling – Shortest Job First

- Problem for Practice

Process	Arrival Time	Burst Time
p1	0	6
p2	0	8
p3	0	7
P3	0	3

Process Scheduling – Round Robin

- It is particularly well-suited for time-sharing systems

Key Features

- **Circular Queue**
  - The ready queue is treated as a circular queue. Each process is placed in the queue in a round-robin fashion.
- **Time Quantum (Time Slice):**
  - A small unit of time, called a time quantum or time slice, is defined. This time quantum is usually in the range of 10-100 milliseconds.

Process Scheduling – Round Robin - Working

- **Process Execution**
  - The CPU scheduler picks the first process from the ready queue and sets a timer to interrupt after one time quantum.
- **Timer Interrupt**
  - If the process completes its execution before the time quantum expires, it is removed from the queue.
  - If the process does not complete within the time quantum, it is interrupted and placed at the end of the ready queue, and the CPU scheduler picks the next process in the queue.
- **Context Switching**
  - A context switch occurs at the end of each time quantum, where the CPU saves the state of the current process and loads the state of the next process.

Process Scheduling – Round Robin - Merit

- **Fairness**
  - Each process gets an equal share of the CPU time, making it a fair scheduling algorithm.
- **Responsiveness**
  - It is particularly effective in time-sharing systems where the goal is to provide a responsive system to the users.
- **Simplicity**
  - The algorithm is easy to implement and understand.

Round Robin - Drawbacks

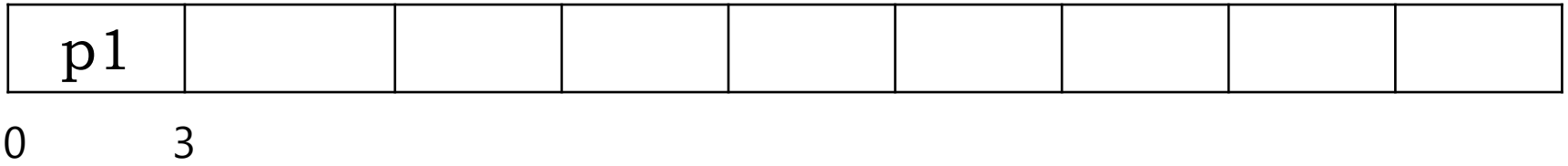
- **Context Switching Overhead**
  - Frequent context switches can lead to high overhead, especially if the time quantum is very small.
- **Performance**
  - If the time quantum is too large, it can behave like First-Come, First-Served (FCFS) scheduling, leading to poor average response time.
  - If the time quantum is too small, it can result in excessive context switching, reducing the overall efficiency

Process Scheduling – Round Robin

- Consider the following processes with their burst times and arrival times: Let the time quantum be 3.

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

- Constructing Gantt

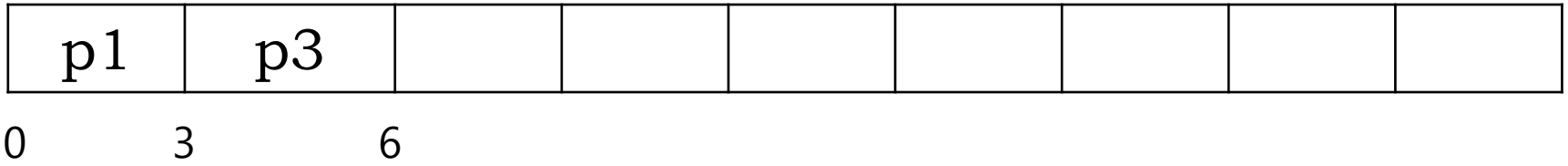


Process Scheduling – Round Robin

- Consider the following processes with their burst times and arrival times: Let the time quantum be 3.

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

- Constructing Gantt

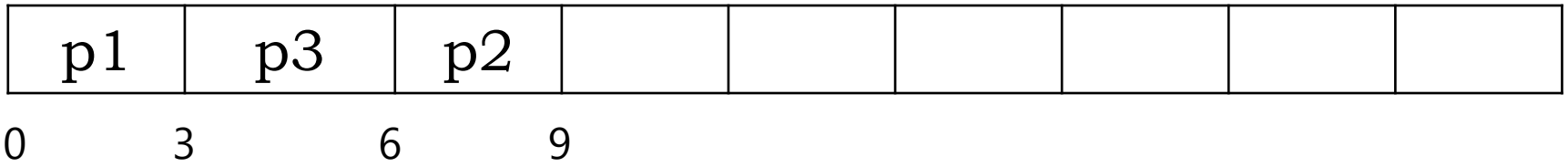


Process Scheduling – Round Robin

- Consider the following processes with their burst times and arrival times: Let the time quantum be 3.

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

- Constructing Gantt

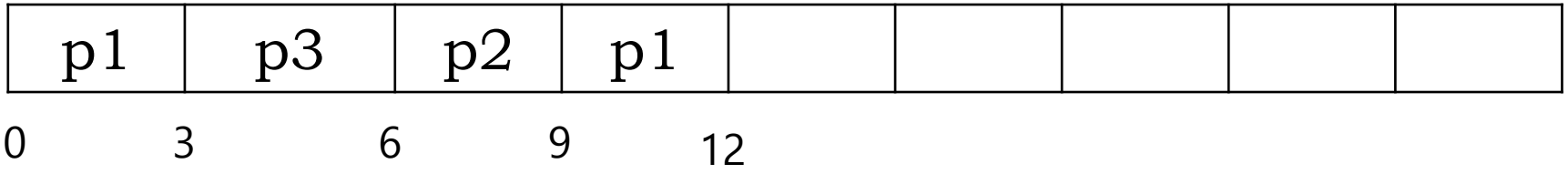


Process Scheduling – Round Robin

- Consider the following processes with their burst times and arrival times: Let the time quantum be 3.

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

- Constructing Gantt

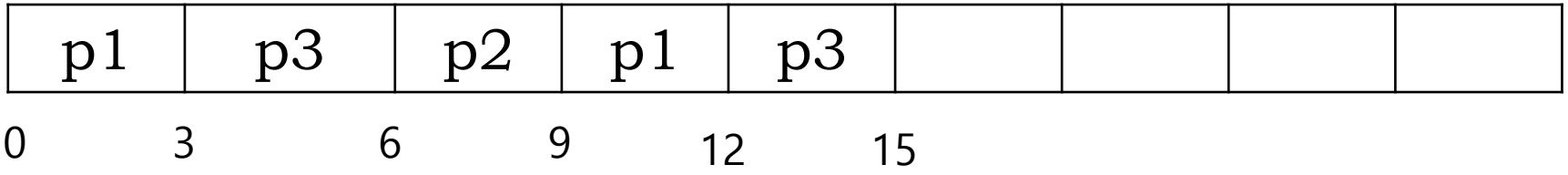


Process Scheduling – Round Robin

- Consider the following processes with their burst times and arrival times: Let the time quantum be 3.

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

- Constructing Gantt

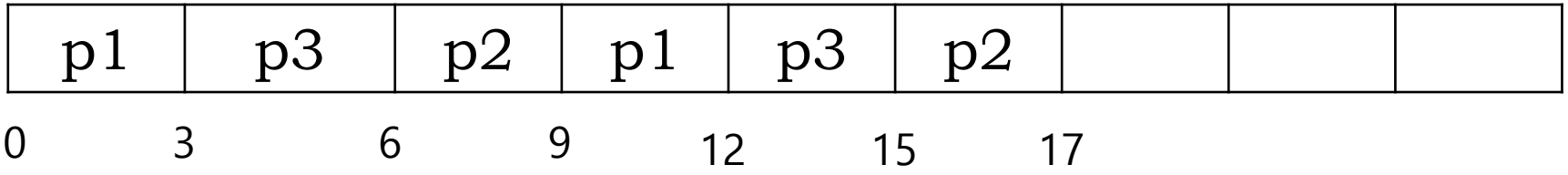


Process Scheduling – Round Robin

- Consider the following processes with their burst times and arrival times: Let the time quantum be 3.

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

- Constructing Gantt

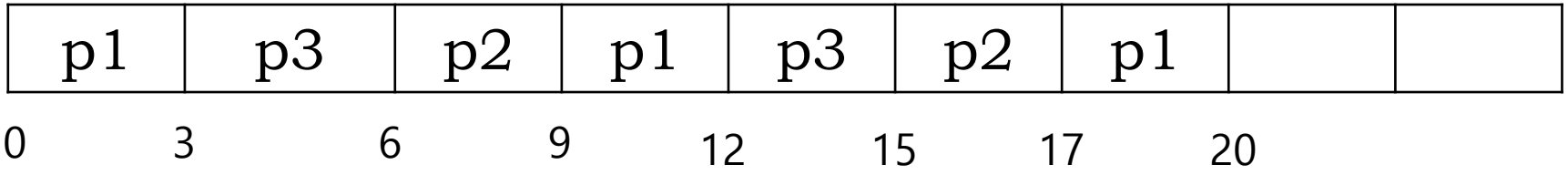


Process Scheduling – Round Robin

- Consider the following processes with their burst times and arrival times: Let the time quantum be 3.

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

- Constructing Gantt

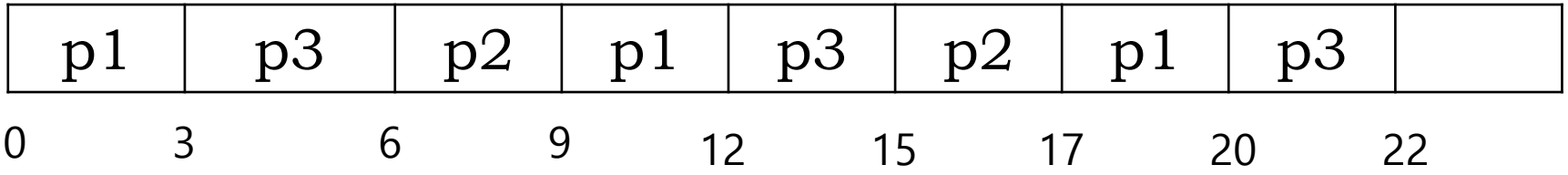


Process Scheduling – Round Robin

- Consider the following processes with their burst times and arrival times: Let the time quantum be 3.

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

- Constructing Gantt

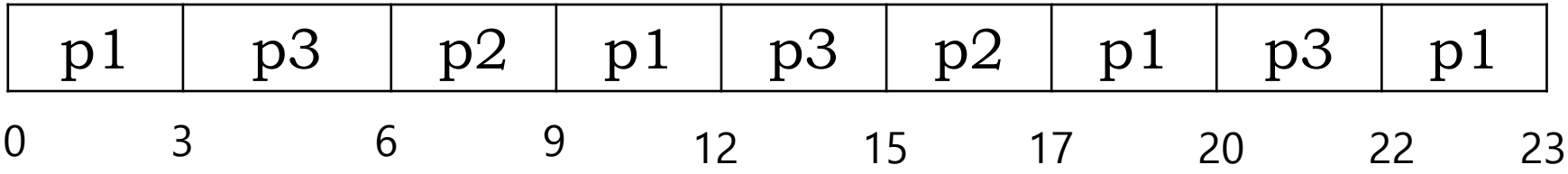


Process Scheduling – Round Robin

- Consider the following processes with their burst times and arrival times: Let the time quantum be 3.

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

- Constructing Gantt



Process Scheduling – Round Robin

Calculations

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
p1	0	10	23	23 – 0 = 23	23 – 10 = 13
p2	2	5	17	17 – 2 = 15	15 – 5 = 10
p3	1	8	22	22 – 1 = 21	21 – 8 = 13
Average Turnaround Time				(23 + 15 + 21)/3 = 19.67	
Average Waiting Time					(13 + 10 + 13)/3 = 12

Process Scheduling – Priority Scheduling

- Each process is assigned a priority, and the CPU is allocated to the process with the highest priority.

- If two processes have the same priority, they are scheduled according to their arrival times (or another secondary criterion).

Process Scheduling – Priority Scheduling

- **Preemptive Priority Scheduling**
  - If a new process arrives with a higher priority than the currently running process, the CPU is preempted and allocated to the new process.
- **Non-Preemptive Priority Scheduling**
  - Once the CPU is allocated to a process, it is not preempted until it completes its burst time, even if a new process with a higher priority arrives.

Process Scheduling – Priority Scheduling

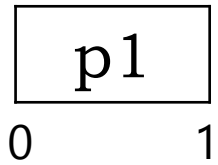
- **Priority Assignment**: Each process is assigned a priority level.
- **Process Selection**: The process with the highest priority is selected for execution.
- **Execution**: The selected process runs until it completes or is preempted by a higher priority process (in preemptive priority scheduling).
- **Next Process**: Once the current process finishes or is preempted, the scheduler selects the next highest priority process from the ready queue

Process Scheduling – Priority Scheduling

- Consider the following processes with their burst times, arrival times, and priorities:

Process	Arrival Time	Burst Time	Priority
p1	0	5	3
p2	1	3	2
p3	2	2	4

- Constructing Gantt - **Preemptive**

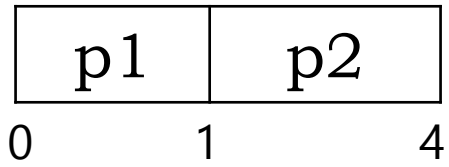


Process Scheduling – Priority Scheduling

- Consider the following processes with their burst times, arrival times, and priorities:

Process	Arrival Time	Burst Time	Priority
p1	0	5	3
p2	1	3	2
p3	2	2	4

- Constructing Gantt - **Preemptive**

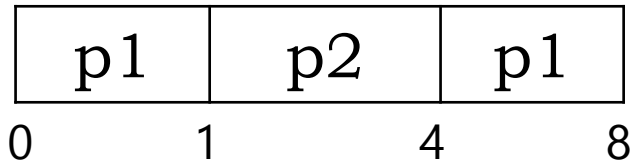


Process Scheduling – Priority Scheduling

- Consider the following processes with their burst times, arrival times, and priorities:

Process	Arrival Time	Burst Time	Priority
p1	0	5	3
p2	1	3	2
p3	2	2	4

- Constructing Gantt - **Preemptive**

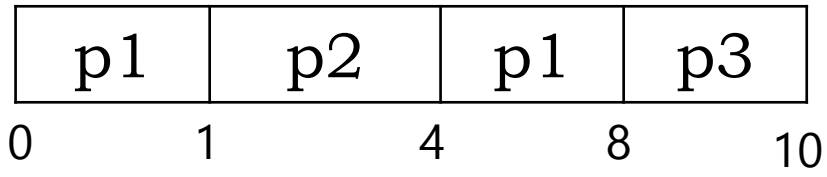


Process Scheduling – Priority Scheduling

- Consider the following processes with their burst times, arrival times, and priorities:

Process	Arrival Time	Burst Time	Priority
p1	0	5	3
p2	1	3	2
p3	2	2	4

- Constructing Gantt - **Preemptive**



Process Scheduling – Priority Scheduling

- Calculations

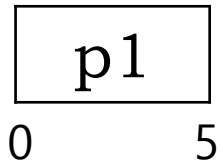
Process	Arrival Time	Burst Time	Priority	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
p1	0	5	3	8	8 – 0 = 8	8 – 5 = 3
p2	1	3	2	4	4 – 1 = 3	3 – 3 = 0
p3	2	2	4	10	10 – 2 = 8	8 – 2 = 6
Average Turnaround Time					(8 + 3 + 8)/3 = 6.33	
Average Waiting Time						(3 + 0 + 6)/3 = 3

Process Scheduling – Priority Scheduling

- Consider the following processes with their burst times, arrival times, and priorities:

Process	Arrival Time	Burst Time	Priority
p1	0	5	3
p2	1	3	2
p3	2	2	4

- Constructing Gantt – **Non-Preemptive**

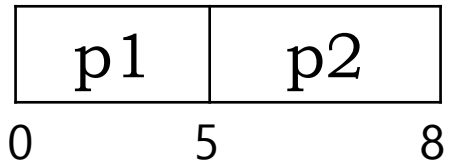


Process Scheduling – Priority Scheduling

- Consider the following processes with their burst times, arrival times, and priorities:

Process	Arrival Time	Burst Time	Priority
p1	0	5	3
p2	1	3	2
p3	2	2	4

- Constructing Gantt - **Non-Preemptive**

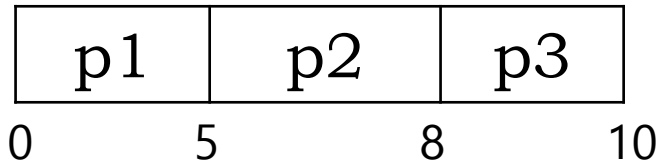


Process Scheduling – Priority Scheduling

- Consider the following processes with their burst times, arrival times, and priorities:

Process	Arrival Time	Burst Time	Priority
p1	0	5	3
p2	1	3	2
p3	2	2	4

- Constructing Gantt - **Non-Preemptive**



Process Scheduling – Priority Scheduling

- Calculations

Process	Arrival Time	Burst Time	Priority	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
p1	0	5	3	5	5 – 0 = 5	5 – 5 = 0
p2	1	3	2	8	8 – 1 = 7	7 – 3 = 4
p3	2	2	4	10	10 – 2 = 8	8 – 2 = 6
Average Turnaround Time					(5 + 7 + 8)/3 = 6.67	
Average Waiting Time						(0 + 4 + 6)/3 = 3.33

Problems for Practice

Problem 1

Consider the following processes with their burst times and arrival times: Let the time quantum be 3. Compute the average turnaround time and average waiting time

Process	Arrival Time	Burst Time
p1	0	10
p2	2	5
p3	1	8

Problems for Practice

Problem 2

Consider the following processes with their burst times and arrival times. Compute the average turnaround time and average waiting time using Priority scheduling algorithm (Preemptive)

Process	Arrival Time	Burst Time	Priority
p1	0	6	2
p2	2	4	1
p3	2	6	1
p4	0	4	2

Problems for Practice

Problem 3

Consider the following processes with their burst times and arrival times. Compute the average turnaround time and average waiting time using Priority scheduling algorithm (Preemptive)

Process	Arrival Time	Priority	CPU Burst 1	I/O Burst	CPU Burst 2
p1	0	2	4	5	3
p2	1	1	3	-	-
p3	2	3	4	-	-

Problems for Practice

Problem 4

Consider the following processes with their burst times and arrival times. Compute the average turnaround time and average waiting time using Priority scheduling algorithm (Preemptive). Assume lower priority number = higher priority.

Process	Arrival Time	Priority	CPU Burst 1	I/O Burst	CPU Burst 2
p1	0	3	4	3	3
p2	1	1	3	2	2
p3	2	4	2	3	2
p4	3	2	3	2	1
p5	4	5	2	2	2

Process Scheduling – Priority with RR

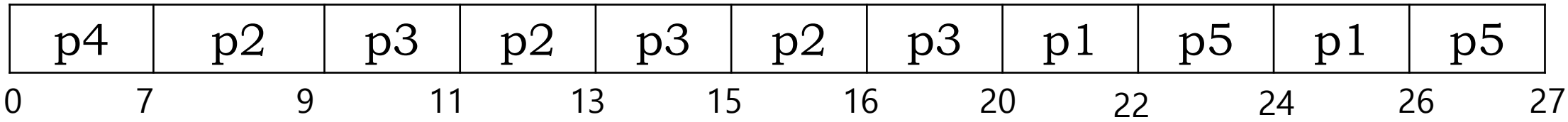
Run the process with the highest priority. Processes with the same priority run round-robin with Tq = 2

Process	Arrival Time	Burst Time	Priority
p1	0	4	3
p2	0	5	2
p3	0	8	2
p4	0	7	1
p5	0	3	3

Process Scheduling – Priority with RR

Gantt Chart

Process	Arrival Time	Burst Time	Priority
p1	0	4	3
p2	0	5	2
p3	0	8	2
p4	0	7	1
p5	0	3	3



Process Scheduling – Priority with RR

Calculations

Process	Arrival Time	Burst Time	Priority	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
p1	0	4	3	26	26 – 0 = 26	26 – 4 = 22
p2	0	5	2	16	16 – 0 = 16	16 – 5 = 11
p3	0	8	2	20	20 – 0 = 20	20 – 8 = 12
p4	0	7	1	7	7 - 0 = 7	7 – 7 = 0
p5	0	3	3	27	27 - 0 = 27	27 – 3 = 24
Average Turnaround Time					19	
Average Waiting Time						13.8